

FutureKawa

Documentation Broker MQTT

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 02/07/2026

Projet : FutureKawa

Sommaire

Broker MQTT — FutureKawa

01 — Vue d'ensemble

02 — Architecture & Listeners

03 — Topics MQTT

04 — Authentification & ACL

05 — Sécurité TLS

06 — Configuration de référence

07 — Tests & validation

Broker MQTT — FutureKawa

Projet	FutureKawa
Document	Documentation du broker MQTT
Version	1.0
Date	02/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Résumé

Le broker MQTT est le **point central de communication** entre les capteurs IoT, le simulateur et l'API d'exploitation. Il repose sur **Eclipse Mosquitto 2.0**, déployé via Docker, avec authentification obligatoire et TLS auto-signé.

Composant	Rôle
Mosquitto 2.0	Broker MQTT — routage des messages entre producteurs et consommateurs
<code>entrypoint.sh</code>	Génération des certificats TLS et du fichier de mots de passe au démarrage
<code>mosquitto.conf</code>	Configuration des 4 listeners, TLS, persistance, logs
<code>acl.conf</code>	Contrôle d'accès aux topics par utilisateur

Navigation

#	Document	Contenu
1	Vue d'ensemble	Rôle, choix technique, flux global

#	Document	Contenu
2	Architecture & listeners	Ports, TLS, Docker, réseau
3	Topics MQTT	Hiérarchie, payloads, conventions
4	Authentification & ACL	Utilisateurs, règles d'accès
5	Sécurité TLS	Certificats, génération, recommandations prod
6	Configuration de référence	Explication ligne par ligne de mosquitto.conf
7	Tests & validation	Script test_acl.sh, procédures de vérification

01 — Vue d'ensemble

[Suivant : Architecture & listeners →](#)

Rôle dans l'architecture FutureKawa

Le broker est le **bus de messagerie** du système. Il découple les producteurs (capteurs IoT, simulateur) des consommateurs (API d'exploitation, frontend) sans qu'aucun d'eux ne connaisse l'autre.

```
Capteur IoT (ESP32)    → MQTT → Broker → MQTT →  
mqtt_subscriber.py (API)  
Simulateur (Python)   → MQTT → Broker → MQTT → Frontend  
(WebSocket)
```

Sans le broker : - Chaque capteur devrait connaître l'adresse de l'API -
L'API devrait gérer N connexions simultanées - Le frontend ne pourrait pas recevoir les données en temps réel sans polling

Choix technique : Eclipse Mosquitto

Critère	Mosquitto	Alternative RabbitMQ	Alternative Kafka
Protocole	MQTT natif	AMQP (plugin MQTT)	Kafka (pas MQTT)
Empreinte mémoire	< 5 Mo	> 100 Mo	> 500 Mo
Adapté IoT	Oui	Partiel	Non
TLS intégré	Oui	Oui	Oui
WebSocket	Oui	Oui	Non natif
Complexité opérationnelle	Faible	Moyenne	Haute

Mosquitto est la référence pour les projets IoT de taille intermédiaire : léger, fiable, supporte nativement MQTT 3.1.1 et 5.0, WebSocket et TLS.

Flux de données

```
[ESP32 / Simulateur]
|
| publish capteur/iot/temperature {"metric": "temperature",
"value": 22.4, ...}
| publish capteur/iot/humidity {"metric": "humidity",
"value": 61.2, ...}
|
v
[Broker Mosquitto]
|
|-- subscribe capteur/# → [mqtt_subscriber.py]
|
|
|
|
|
|
|-- subscribe capteur/# → [Frontend WebSocket]
|
|
v
Affichage temps réel
```

Acteurs

Acteur	Rôle MQTT	Topics
Capteur IoT (ESP32)	Producteur	capteur/iot/ temperature, capteur/ iot/humidity, capteur/iot/error
Simulateur Python	Producteur	capteur/simu-sensor/ temperature, capteur/ simu-sensor/humidity
mqtt_subscriber.py	Consommateur	capteur/# (tous)

Acteur	Rôle MQTT	Topics
Frontend Vue.js	Consommateur WebSocket	capteur/#

[Suivant : Architecture & listeners →](#)

02 — Architecture & Listeners

Les 4 listeners

Le broker expose 4 points d'entrée distincts selon le contexte (réseau interne Docker vs exposition publique, MQTT vs WebSocket).

Port	Protocole	TLS	Usage
1883	MQTT	Non	Communication interne Docker (API, simulateur)
9001	MQTT WebSocket	Non	Frontend sur réseau Docker interne
8883	MQTT	Oui	Capteurs IoT externes (Internet)
9443	MQTT WebSocket	Oui	Frontend exposé publiquement

Règle de sécurité réseau

Les ports 1883 et 9001 (sans TLS) **ne sont jamais exposés hors du réseau Docker**. Seuls 8883 et 9443 sont ouverts sur l'interface publique.

```
Internet
|
|-- 8883 (MQTT+TLS)  → Capteurs IoT ESP32
|-- 9443 (WS+TLS)   → Frontend navigateur
|
[Docker network]
|-- 1883 (MQTT)      → mqtt_subscriber.py, simulateur
|-- 9001 (WS)        → Frontend sur réseau interne (dev)
```


Dockerfile

```
FROM eclipse-mosquitto:2.0

RUN apk add --no-cache openssl

COPY mosquitto.conf /mosquitto/config/mosquitto.conf
COPY acl.conf /mosquitto/config/acl.conf
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 1883 9001 8883 9443

ENTRYPOINT ["/entrypoint.sh"]
```

Le paquet `openssl` est ajouté à l'image Alpine pour la génération des certificats TLS au démarrage.

Entrypoint

À chaque démarrage du conteneur, `entrypoint.sh` :

1. Crée le fichier `passwd` avec les credentials depuis les variables d'environnement
2. Vérifie si les certificats TLS existent — si non, les génère automatiquement (CA + serveur)
3. Lance Mosquitto avec la configuration

```
mosquitto_passwd -b -c /mosquitto/config/passwd "$MQTT_USER"
"$MQTT_PASSWORD"
```

Les certificats sont générés dans `/mosquitto/certs/` et persistent grâce au volume Docker.

Variables d'environnement requises

Variable	Description	Exemple
<code>MQTT_USER</code>	Nom d'utilisateur MQTT	<code>adminFutureKawa</code>

Variable	Description	Exemple
MQTT_PASSWORD	Mot de passe MQTT	<secret>

Volumes Docker

Volume	Contenu	Raison
/mosquitto/data/	Messages persistés	Survie aux redémarrages
/mosquitto/certs/	Certificats TLS	Régénération évitée à chaque restart
/mosquitto/config/	Fichiers de config	Écriture du fichier <code>passwd</code> à l'init

Réseau Docker

Le broker est sur le même réseau Docker que `API-exploitation`. Le nom de service `broker` est résolu automatiquement par Docker DNS — c'est pourquoi `mqtt_subscriber.py` utilise `MQTT_BROKER=broker` par défaut.

03 — Topics MQTT

Hiérarchie des topics

Tous les topics suivent la convention `capteur/<source>/<métrique>`.

```
capteur/  
├─ iot/  
│   ├─ temperature    ← Vrai capteur ESP32 → température réelle  
│   ├─ humidity       ← Vrai capteur ESP32 → humidité réelle  
│   └─ error          ← Erreur DHT22 / capteur défaillant  
└─ simu-sensor/  
    ├─ temperature    ← Simulateur Python → température fictive  
    └─ humidity       ← Simulateur Python → humidité fictive  
  
health/  
└─ ping              ← Topic réservé pour health checks
```

Topics de mesure

`capteur/iot/temperature` **et** `capteur/simu-sensor/temperature`

Publié par le capteur IoT ou le simulateur à chaque mesure de température.

Payload JSON :

```
{  
  "metric": "temperature",  
  "value": 22.4,  
  "timestamp": "2026-07-02T14:32:10.123Z",  
  "sensor_id": "capteur-zone-A"  
}
```

Champ	Type	Description
<code>metric</code>	string	Toujours <code>"temperature"</code>
<code>value</code>	float	Valeur en °C
<code>timestamp</code>	string ISO 8601	Horodatage UTC
<code>sensor_id</code>	string	Identifiant unique du capteur

`capteur/iot/humidity` **et** `capteur/simu-sensor/humidity`

Même structure, `metric` vaut `"humidity"`, `value` est en %.

Règle de rejet

Le subscriber rejette les valeurs égales à 0 ou quasi-nulles (`-0.1 < value < 0.1`) — signature d'un DHT22 défaillant.

Topic d'erreur capteur

`capteur/iot/error`

Publié par l'ESP32 quand il détecte plusieurs lectures consécutives invalides.

Payload JSON :

```
{
  "error_type": "DHT_READ_FAILURE",
  "consecutive_errors": 5,
  "message": "Lecture DHT22 échouée 5 fois de suite",
  "sensor_id": "capteur-zone-A",
  "timestamp": "2026-07-02T14:35:00.000Z"
}
```

Champ	Type	Description
<code>error_type</code>	string	Type d'erreur (<code>DHT_READ_FAILURE</code> , <code>UNKNOWN</code> , ...)

Champ	Type	Description
consecutive_errors	int	Nombre d'échecs consécutifs
message	string	Message lisible
sensor_id	string	Capteur concerné

À la réception, le subscriber : 1. Marque le capteur `is_active = False` en base 2. Crée une alerte de type `Capteur défaillant`

Wildcards disponibles

Wildcard	Correspond à	Usage
<code>capteur/#</code>	Tous les topics sous <code>capteur/</code>	Subscription globale du subscriber
<code>capteur+/temperature</code>	<code>capteur/iot/temperature</code> ET <code>capteur/simu-sensor/temperature</code>	Subscription ciblée température
<code>capteur+/humidity</code>	<code>capteur/iot/humidity</code> ET <code>capteur/simu-sensor/humidity</code>	Subscription ciblée humidité

Buffer et agrégation

Le subscriber ne sauvegarde une mesure en base **que si les deux métriques** (température ET humidité) sont disponibles pour le même capteur dans une fenêtre de 2 secondes. Cela évite les enregistrements partiels.

Réception `capteur/iot/temperature` → mise en buffer

Réception `capteur/iot/humidity` → buffer complet → save en DB

Si une seule métrique arrive dans les 2 secondes, le buffer est vidé et la mesure incomplète ignorée.

Limites configurées

Paramètre	Valeur	Description
<code>message_size_limit</code>	65 536 octets	Taille max d'un message
<code>max_queued_messages</code>	100	File d'attente max par client
<code>persistent_client_expiration</code>	1 h	Durée de vie d'une session persistante

04 — Authentication & ACL

Authentication par mot de passe

L'accès anonyme est interdit sur tous les listeners :

```
allow_anonymous false
```

Mosquitto utilise un fichier `passwd` généré au démarrage par `mosquitto_passwd`. Les credentials transitent via les variables d'environnement `MQTT_USER` et `MQTT_PASSWORD` — jamais stockés en clair dans les fichiers de config versionnés.

Génération du fichier passwd

```
mosquitto_passwd -b -c /mosquitto/config/passwd "$MQTT_USER"  
"$MQTT_PASSWORD"
```

Le flag `-b` (batch) permet de passer le mot de passe en argument. Le flag `-c` crée le fichier (écrase s'il existe). Le mot de passe est stocké haché (PBKDF2-SHA512) dans le fichier `passwd`.

Contrôle d'accès (ACL)

Le fichier `acl.conf` définit les droits de chaque utilisateur sur les topics.

Configuration actuelle

```
user adminFutureKawa  
topic readwrite capteur/#  
topic readwrite health/#
```

Utilisateur	Permission	Topics
adminFutureKawa	readwrite	capteur/# , health/#

Niveaux d'accès disponibles

Niveau	Description
read	Abonnement uniquement (subscribe)
write	Publication uniquement (publish)
readwrite	Les deux
deny	Accès interdit

Architecture multi-utilisateurs (évolution recommandée)

L'architecture actuelle utilise un seul compte pour tous les acteurs. En production, il est recommandé de créer des comptes distincts avec des droits réduits :

Utilisateur	Rôle	Permission	Topics
sensor-iot	Capteurs ESP32	write	capteur/iot/#
simu-sensor	Simulateur	write	capteur/simu-sensor/#
api-subscriber	mqtt_subscriber.py	read	capteur/#
frontend	Frontend Vue.js	read	capteur/#
admin	Monitoring, health	readwrite	#

Cela applique le **principe du moindre privilège** : un capteur compromis ne peut pas lire les messages des autres ni publier sur des topics non autorisés.

Exemple d'acl.conf multi-utilisateurs

```
user sensor-iot
topic write capteur/iot/#

user simu-sensor
topic write capteur/simu-sensor/#
```



```
user api-subscriber
topic read capteur/#

user frontend
topic read capteur/#

user admin
topic readwrite #
```

Ajout d'un utilisateur

Pour ajouter un utilisateur sans recréer le fichier :

```
# Depuis l'hôte, avec le conteneur en cours d'exécution
docker exec broker mosquitto_passwd /mosquitto/config/passwd
<nouveau_user>

# Recharger la config (sans redémarrer)
docker exec broker mosquitto_pub -h localhost -p 1883 \
  -u "$MQTT_USER" -P "$MQTT_PASSWORD" \
  -t '$SYS/broker/control' -m ''
```

Ou via un redémarrage du conteneur (l'entrypoint recrée le fichier).

05 — Sécurité TLS

Principe

Les listeners `8883` (MQTT) et `9443` (WebSocket) requièrent TLS 1.2 minimum. Les données capteur (température, humidité) transitent chiffrées sur Internet — un attaquant ne peut pas intercepter ni injecter de fausses mesures.

Certificats auto-signés (développement)

À chaque démarrage, si les certificats sont absents, `entrypoint.sh` les génère automatiquement :

Étapes de génération

```
# 1. Autorité de certification (CA) – clé 4096 bits, validité 10 ans
openssl genrsa -out ca.key 4096
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt \
    -subj "/CN=FutureKawa-CA/O=FutureKawa/C=FR"

# 2. Clé serveur + CSR
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr \
    -subj "/CN=broker/O=FutureKawa/C=FR"

# 3. Certificat serveur signé par la CA
openssl x509 -req -days 3650 \
    -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -out server.crt
```

Fichiers générés

Fichier	Rôle	Permissions
<code>ca.key</code>	Clé privée de la CA	640 (mosquitto seul)

Fichier	Rôle	Permissions
ca.crt	Certificat de la CA (distribué aux clients)	644
server.key	Clé privée du serveur	640 (mosquitto seul)
server.crt	Certificat du serveur	644

Persistence

Les certificats sont stockés dans le volume Docker `/mosquitto/certs/`. Si le volume persiste entre les redémarrages, les certificats ne sont pas régénérés — comportement contrôlé par le test `if [ ! -f "$CERT_DIR/server.crt" ]`.

Configuration TLS dans mosquitto.conf

```
listener 8883
cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key
tls_version tlsv1.2
```

`tls_version tlsv1.2` interdit TLS 1.0 et 1.1, considérés obsolètes.

Connexion cliente avec certificat auto-signé

Les clients (ESP32, Python) doivent disposer du `ca.crt` pour valider le certificat serveur.

Python (paho-mqtt)

```
import ssl
import paho.mqtt.client as mqtt

client = mqtt.Client()
client.tls_set(
    ca_certs="ca.crt",
    tls_version=ssl.PROTOCOL_TLSv1_2
```

```
)  
client.connect("broker.futurekawa.local", 8883)
```

Arduino / ESP32

```
WiFiClientSecure espClient;  
espClient.setCACert(ca_cert); // ca.crt en constante char*  
PubSubClient client(espClient);  
client.setServer("broker.futurekawa.local", 8883);
```

Recommandations production

Mesure	Priorité	Description
Certificat Let's Encrypt	Haute	Remplacer les certificats auto-signés par un certificat signé par une CA publique
CN = nom de domaine réel	Haute	Le CN <code>broker</code> doit correspondre au FQDN exposé
Rotation des mots de passe	Haute	<code>MQTT_PASSWORD</code> à changer régulièrement, hors dépôt git
Multi-utilisateurs ACL	Moyenne	Voir section Authentification & ACL
Monitoring TLS	Faible	Alerter sur l'expiration des certificats (validité 3650 j)

06 — Configuration de référence

Explication ligne par ligne de `broker/mosquitto.conf`.

Listener MQTT interne (port 1883)

```
listener 1883
allow_anonymous false
```

Port standard MQTT, sans TLS. Accessible uniquement sur le réseau Docker interne. `allow_anonymous false` est répété par listener pour être explicite — Mosquitto applique la règle globale, mais la répétition évite les ambiguïtés lors d'une lecture rapide.

Listener WebSocket interne (port 9001)

```
listener 9001
protocol websockets
allow_anonymous false
```

WebSocket MQTT pour le frontend sur réseau Docker interne (environnement de développement). Le navigateur ne supporte pas le protocole TCP MQTT natif — il faut WebSocket.

Listener MQTT + TLS (port 8883)

```
listener 8883
cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key
tls_version tlsv1.2
allow_anonymous false
```

Point d'entrée pour les capteurs IoT exposés sur Internet. TLS 1.2 obligatoire. Les certificats sont générés au démarrage si absents.

Listener WebSocket + TLS (port 9443)

```
listener 9443
protocol websockets
cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key
tls_version tlsv1.2
allow_anonymous false
```

WebSocket sécurisé pour le frontend exposé sur Internet. Même configuration TLS que 8883.

Authentication & ACL

```
password_file /mosquitto/config/passwd
acl_file /mosquitto/config/acl.conf
```

`passwd` est généré au démarrage par `entrypoint.sh`. `acl.conf` est copié depuis le dépôt à la construction de l'image.

Limites et sécurité des messages

```
message_size_limit 65536
max_queued_messages 100
persistent_client_expiration 1h
```

Paramètre	Valeur	Justification
<code>message_size_limit</code>	65 536 octets	Suffisant pour les payloads JSON capteur (< 200 octets), bloque les abus

Paramètre	Valeur	Justification
<code>max_queued_messages</code>	100	Évite qu'un client lent sature la mémoire du broker
<code>persistent_client_expiration</code>	1 h	Nettoie les sessions persistantes après 1 h d'inactivité

Persistence

```
persistence          true
persistence_location /mosquitto/data/
```

Mosquitto persiste les messages QoS 1 et 2 en attente sur disque. En cas de redémarrage, les messages non acquittés sont conservés.

Logs

```
log_dest      stdout
log_type      error
log_type      warning
log_type      notice
log_type      information
log_timestamp  true
log_timestamp_format %Y-%m-%dT%H:%M:%S
```

Les logs sont envoyés sur `stdout` (récupérés par Docker). Les 4 niveaux sont activés en développement — en production, `error` et `warning` suffisent pour réduire le volume.

Format de timestamp ISO 8601 pour la cohérence avec les timestamps des payloads.

Tableau de synthèse

Paramètre	Valeur	Impact
<code>allow_anonymous</code>	<code>false</code> (tous listeners)	Connexion impossible sans credentials
<code>tls_version</code>	<code>tlsv1.2</code> (8883, 9443)	TLS 1.0/1.1 refusés
<code>message_size_limit</code>	65 536	Protection contre les gros messages
<code>max_queued_messages</code>	100	Protection mémoire
<code>persistence</code>	<code>true</code>	Survivance aux redémarrages
<code>log_type</code>	error + warning + notice + info	Logs complets (dev)

07 — Tests & validation

Script test_acl.sh

Le fichier `broker/tests/test_acl.sh` est un script shell qui valide la configuration ACL du broker en testant toutes les combinaisons autorisées / refusées.

Prérequis

```
# Paquet mosquitto-clients (Ubuntu/Debian)
sudo apt install mosquitto-clients

# macOS
brew install mosquitto
```

Lancement

```
# Broker local sur port 1883
MQTT_USER=adminFutureKawa MQTT_PASSWORD=<password> ./broker/tests/
test_acl.sh

# Broker distant
MQTT_USER=adminFutureKawa MQTT_PASSWORD=<password> ./broker/tests/
test_acl.sh broker.futurekawa.local 8883
```

Cas de test couverts

Test	Utilisateur	Topic	Action	Attendu
Publication simu temp	adminFuture Kawa	capteur/ simu/temp	publish	OK
Publication simu humi	adminFuture Kawa	capteur/ simu/humi	publish	OK
Publication IoT temp	adminFuture Kawa	capteur/iot/ temp	publish	OK

Test	Utilisateur	Topic	Action	Attendu
Publication IoT humi	adminFuture Kawa	capteur/iot/humi	publish	OK
Abonnement capteur	adminFuture Kawa	capteur/#	subscribe	OK
Publication anonyme	—	capteur/simu/temp	publish	Refusé
Abonnement anonyme	—	capteur/#	subscribe	Refusé
Mauvais mot de passe	adminFuture Kawa	capteur/simu/temp	publish	Refusé

Format de sortie

```
=== Tests ACL Mosquitto – localhost:1883 ===
```

```
Publication (utilisateur autorisé) ---
```

```
[OK] Pub capteur/simu/temp (admin → OK)
[OK] Pub capteur/iot/temp (admin → OK)
[OK] Pub capteur/simu/humi (admin → OK)
[OK] Pub capteur/iot/humi (admin → OK)
```

```
Abonnement (utilisateur autorisé) ---
```

```
[OK] Sub capteur/# (admin → OK)
```

```
Accès anonyme (doit être refusé) ---
```

```
[OK] Pub anonyme capteur/simu/temp (anon → KO attendu)
[OK] Sub anonyme capteur/# (anon → KO attendu)
```

```
Mauvais mot de passe (doit être refusé) ---
```

```
[OK] Pub bad password capteur/simu/temp (badpwd → KO attendu)
```

=== Résultats : 8 OK / 0 KO ===

Tests manuels rapides

Vérifier la connexion au broker

```
# Ping via publication sur topic health
mosquitto_pub -h localhost -p 1883 \
  -u adminFutureKawa -P <password> \
  -t health/ping -m "ok"

# Écouter tous les messages capteur
mosquitto_sub -h localhost -p 1883 \
  -u adminFutureKawa -P <password> \
  -t "capteur/#" -v
```

Simuler un capteur IoT

```
mosquitto_pub -h localhost -p 1883 \
  -u adminFutureKawa -P <password> \
  -t capteur/iot/temperature \
  -m '{"metric":"temperature","value":
22.4,"timestamp":"2026-07-02T14:00:00Z","sensor_id":"test-sensor"}'
```

Tester TLS (port 8883)

```
mosquitto_pub -h localhost -p 8883 \
  -u adminFutureKawa -P <password> \
  --cafile /mosquitto/certs/ca.crt \
  -t health/ping -m "tls-ok"
```

Tests via Docker

```
# Lancer les tests depuis l'intérieur du conteneur broker
docker exec broker sh -c \
  "MQTT_USER=adminFutureKawa MQTT_PASSWORD=<password> \
```

```
/bin/sh /mosquitto/tests/test_acl.sh localhost 1883"
```

```
# Via docker-compose
```

```
docker compose -f docker-compose.exploit.yml exec broker \  
sh /mosquitto/tests/test_acl.sh localhost 1883
```

Surveillance en production

Vérification	Commande	Fréquence
Broker actif	<code>docker compose ps broker</code>	Monitoring continu
Logs d'erreur	<code>docker compose logs broker --since 1h</code>	Quotidien
Test ACL	<code>./broker/tests/test_acl.sh</code>	Avant chaque déploiement
Expiration TLS	<code>openssl x509 -in ca.crt -noout -dates</code>	Mensuel